

# UT4. Desarrollo de aplicaciones web con PHP

---

DESARROLLO WEB EN ENTORNO SERVIDOR (DWS)

A solid orange horizontal bar at the bottom of the slide.

# Contenidos

---

1. Autenticación de usuarios y control de acceso.
2. Cookies.
3. Manejo de sesiones.
4. Herramientas para depuración de código.

# Autenticación de usuarios y control de acceso.

---



Autenticación:

**Paso de parámetros por la URL:**

[http://localhost/hola\\_nombre.php?nombre=Ana](http://localhost/hola_nombre.php?nombre=Ana)

[http://localhost/hola\\_nombre.php?nombre=Ana&apellido=Luna](http://localhost/hola_nombre.php?nombre=Ana&apellido=Luna)

Se recogen en el array **\$\_GET**

```
<?php echo "Hola ". $_GET["nombre"] . " " . "$_GET["apellido"] ?>
```

Si no existe el parámetro, se genera un aviso (tipo warning) **E\_NOTICE** y la ejecución continúa.

# Autenticación de usuarios y control de acceso.

---



Autenticación:

## Paso de parámetros ocultos por POST:

- Fichero **login.php** en el que introducimos el usuario y contraseña.
- Fichero **validate.php** en el que se comprueba el usuario y contraseña introducidos.
- Fichero **welcome.php** en el que se da la bienvenida al usuario y cierra la sesión.
- Fichero **logout.php** cierra la ejecución de la aplicación.
- Usos de seguridad aplicadas:
  - Cifrado de contraseñas en php: `password_hash()` y `password_verify()`
  - SQL Injection: `prepare()` y `bind_param()`
  - Evitar robos de sesión: `session_regenerate_id(true)`
  - Validación y limpieza: `trim()` y `htmlspecialchars()` para evitar XSS.
  - HTTPS siempre recomendado en producción.
  - Bloqueo de accesos a páginas sin acceso login.



# Autenticación de usuarios y control de acceso.

---

## Otros métodos de autenticación (HTTP básica):

### 1. Autenticación a nivel de servidor (creando usuarios y recursos restringidos en Apache):

- **htpasswd**: utilidad que permite almacenar en un fichero (normalmente en la ruta apache2/users) - no accesible vía web – una lista de usuarios y contraseñas cifradas.
- **.htaccess**: fichero (normalmente en la ruta apache2/users) que le indica al servidor de apache los recursos con acceso restringidos (usuarios creados con htpasswd).

### 2. Autenticación a nivel de PHP (usando cabecera HTTP):

- Uso de la variable global `$_SERVER`.
  - `$_SERVER['PHP_AUTH_USER']`: Nombre del usuario que se ha introducido.
  - `$_SERVER['PHP_AUTH_PW']`: Contraseña introducida
  - `$_SERVER['AUTH_TYPE']`: Método HTTP usado para autenticar (Basic o Digest).



# Autenticación de usuarios y control de acceso.

---

## Otros métodos de autenticación (HTTP básica):

- Se hará uso de **validate\_phpServer.php** para comprobar el uso de este método de autenticación.
  - La primera vez que se ejecuta el validate\_phpServer.php el navegador muestra una ventana emergente pidiendo un usuario y contraseña (no es formulario HTML realizado por nosotros en la aplicación).
  - Cuando el usuario ingresa esas credenciales, el navegador las envía por cabecera HTTP.
  - PHP recoge esa cabecera y las coloca en las variables \$\_SERVER.
  - El código desarrollado puede validar las credenciales.
- Consideraciones **IMPORTANTES** sobre seguridad:
  - Esta autenticación no cifra la contraseña sólo la codifica, por ello es obligatorio el uso de HTTPS.
  - No se deben guardar las contraseñas en texto plano (idealmente usar una base de datos con password\_hash() o cualquier método de encriptación).
  - En algunos servidores web de las plataformas vistas en clase habrán que realizar modificaciones para que funcione correctamente, por ejemplo en CGI.



# Cookies

---

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

# Cookies

---

## Cookies

- Las cookies son pequeños ficheros que dejan los servidores web en los ordenadores de los clientes.
- Cuando un cliente realiza una petición web, envía al servidor las cookies que pudiera tener de este.
- Para manejar las cookies se utiliza la función **setcookie()**

```
bool setcookie (string $name, [, string $value = "" [, int $expire = 0 [, string $path = ""  
[, string $domain = "" [, bool $secure = FALSE [, bool $httponly = FALSE ]]]]])
```

- \$name: nombre de la cookie.
- \$value: valor que se le quiere dar.
- \$expire: fecha en la que expira la cookie (se expresa en segundos). Normalmente se usa la función time() que devuelve la fecha actual.

Por ejemplo:

```
setcookie('visitas', '1', time() + 3600 * 24);
```

Genera una cookie llamada 'visitas' cuyo valor es '1' por un tiempo de vida de 24h.





# Cookies

---

## Cookies

- Las cookies están disponibles en **\$\_COOKIE**.
- La clave es el nombre que se le dio con **setcookie()**.
- Después de crear una cookie esta no está disponible en **\$\_COOKIE** hasta la siguiente petición del cliente.

Por ejemplo (daría un error – **cookie-mal.php**):

```
setcookie('nueva', "valor", time() + 3600 * 24);  
echo $_COOKIE["nueva"];
```

Las cookies se envían como cabeceras de peticiones HTTP. Hay que enviar las cabeceras antes de empezar con el cuerpo de la respuesta. Esto implica que hay que utilizar la función **setcookie()** antes de que se empiece a escribir la salida. Si se intenta llamar a **setcookie** después de haber realizado un **echo**, se producirá un error.



# Cookies

---

## Cookies. Ejemplo <contador\_visitas.php>:

```
<?php
/*buscamos la cookie "visitas" si no existe se crea con valor 1
si existe, se le suma 1. En los dos casos se muestra un mensaje apropiado
*/
if (!isset($_COOKIE['visitas'])){
    setcookie('visitas', '1', time() + 3600 * 24);
    echo "Bienvenido por primera vez";
} else {
    $visitas = (int) $_COOKIE['visitas'];
    $visitas++;
    setcookie('visitas', $visitas, time() + 3600 * 24);
    echo "Bienvenido por $visitas vez";
}
```



# Manejo de sesiones

---

UT04. DESARROLLO DE APLICACIONES WEB CON PHP

# Manejo de sesiones

---



## Sesiones

- Como HTTP es un protocolo sin estado, las diferentes peticiones de un cliente al servidor son independientes.
- Al iniciar una sesión el servidor asigna y envía al usuario un identificador de sesión.
- Para controlar la sesión el servidor deja una cookie con el id de sesión en el cliente, que se elimina al cerrarla.



# Manejo de sesiones

---



## Sesiones

- Para crear una sesión se utiliza la función **start\_session()**.
- Si no hay una sesión activa, la crea. Si la hay, el script que llama a la función se une a ella.
- Una vez creada la sesión es posible utilizar la variable superglobal **\$\_SESSION** para compartir información entre los scripts que comparten sesión.

**`$_SESSION['nombre'] = valor;`**

# Manejo de sesiones



## Sesiones.

- Para cerrar la sesión no basta con llamar a **session\_destroy()**. También hay que borrar las variables de sesión y la cookie (archivo **session\_destroy.php**).

<?php

```
session_start(); // unirse a la sesión
// Borrar todos los elementos almacenados de $_SESSION
$_SESSION = array();
session_destroy(); // eliminar la sesion
setcookie(session_name(), 123, time() - 1000); // eliminar la cookie
// redirigimos a login.php
header("Location: login.php");
```



# Manejo de sesiones



## Sesiones.

### Sesiones\_uso\_basico1.php

```
<?php
    session_start();
    if (!isset($_SESSION['count'])) {
        $_SESSION['count'] = 0;
    } else {
        $_SESSION['count']++;
    }
    echo "hola " . $_SESSION['count'];
    echo "<br><a href='sesiones_uso_basico2.php'>Siguiente</a>";
```

### Sesiones\_uso\_basico2.php

```
<?php
    session_start();
    echo "La variable count vale: " . $_SESSION['count'];
```



# Herramientas para la depuración de código

---

UT04. DESARROLLO DE APLICACIONES WEB CON PHP





# Herramientas para la depuración del código

---

- PHP incluye un depurador a partir de la versión 5.6, aunque es habitual el uso de depuradores externos integrados en un IDE.
- Se pueden utilizar los `echo`, `print`, `print_r`, `var_dump`, `die($var)` o `exit` para realizar depuraciones. Pero este método resulta tedioso y requiere de cambios constantes en el código de la aplicación. Por ejemplo, si tenemos la variable `$_SERVER` podemos introducir la siguientes líneas para comprobar su valor:

```
<?php
var_dump($_SERVER);    // Permite ver el tipo y contenido exacto de una variable
echo "<br>";
print_r($_SERVER);     // Muestra solo el valor (sin tipo) – Más legible en tipos como arrays
var_export($_SERVER);  // Muestra la estructura tal como PHP la utilizaría (usado para exportar)
```

- Los depuradores más extendidos son **ZendDebugger** y **Xdebug**.
- **Xdebug**: se instala como una extensión de código libre que se integra perfectamente con Visual Studio Code. Es un servidor que recibe instrucciones de un cliente, por ejemplo VSC.
- Si la depuración está activada, **Xdebug** controla la ejecución.



# Herramientas para la depuración del código

---

## Xdebug.

El proceso de depuración es similar en todos los lenguajes:

- Se establece al menos un punto de ruptura (breakpoint).
- El código se ejecutará hasta el primer breakpoint establecido. A partir de ahí, es posible ejecutar la siguiente línea de código pulsando una tecla, en muchas ocasiones F6.
- Si se trata de una función, pulsando, normalmente F5, se entra en su código.

Como características importantes de **Xdebug** se tienen las siguientes:

- Creación de registros con las llamadas a funciones que se produzcan, incluyendo parámetros y valores devueltos.
- Creación de registros de optimización, que permitan analizar el rendimiento de los programas desarrollados.
- Depuración remota.



# Herramientas para la depuración del código

---

Como características importantes de **Xdebug** se tienen las siguientes:

- Creación de registros con las llamadas a funciones que se produzcan, incluyendo parámetros y valores devueltos.
- Creación de registros de optimización, que permitan analizar el rendimiento de los programas desarrollados.
- Depuración remota.

